CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 02/Unit 02: Arrays

Tauqeer Saleem

Slides based on Dr Faisal Alvi's slides

For any suggested modifications please email: tauqeer.saleem@sse.habib.edu.pk

Unit Outline

1. Review of Last Week's Content
2. The switch-case statement
3. Arrays in C++
4. Arrays and Functions
5. C-Strings

Introduction

- Last week, we covered the basics of C++.
- In particular, the following topics were covered: variables, data types, conditional statements (if-then-else), loops (for-loop, while-loop, do-while-loop).
- We also introduced the increment operator ++?
- One of the questions posed in the class was:
- What is the difference between `++i` and `i++`?

Pre-increment and Post-increment Operators

- `++i` is called the 'pre-increment' operator. `i++` is called the 'post-increment' operator.
- `++i` increments the value of its operand (`i`), before executing the statement.
- So, the output of the snippet:

`int i=4; cout << "i=" << ++i;` is

`i=5;`

- What is the output of the snippet:

`int i=4; cout << "i=" << i++;?`

What is the output of the following program?

```
#include <iostream>
using namespace std;
int main(void) {
    int i = 4;
    cout << "i = " << (i++) << "\n";
    if (++i > 5 )
        cout << "i is equal to 6\n";
    cout << "i = " << i << "\n";
    return 0;
}
```

Recall the if-else-statement in C++

```
// Compare and find the maximum value
if (a >= b && a >= c) {
    max = a;
} else if (b >= a && b >= c) {
    max = b;
} else {
    max = c;
}
```

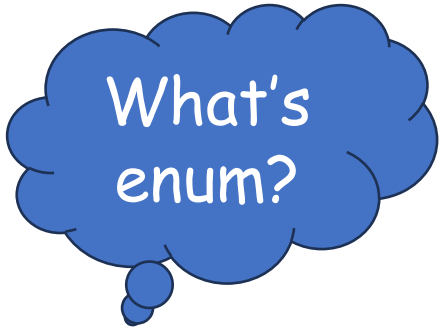
`switch-case` statement

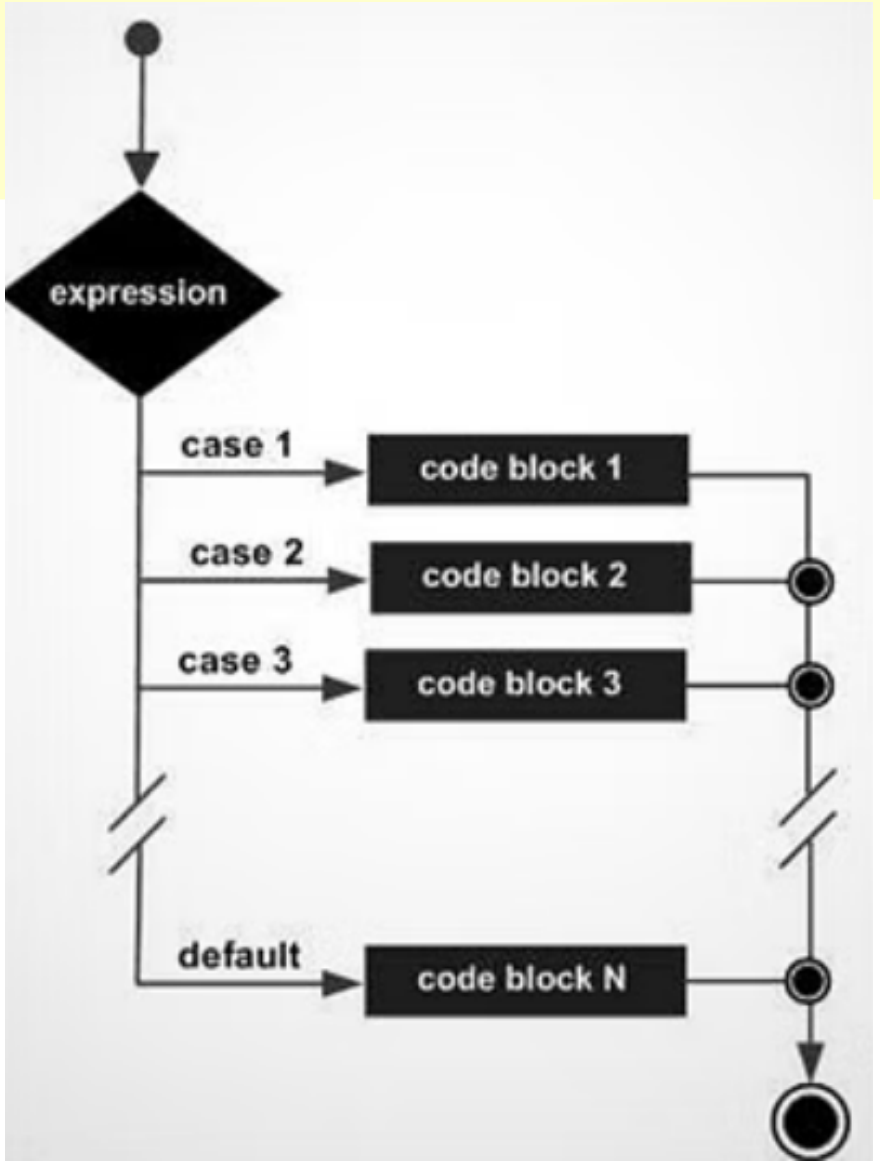
- There is a special construct in C++ that uses multiple if statements.
- This is called the `switch` statement.
- The syntax of the switch-statement is:
  - The switch expression is evaluated once,
  - The value of the expression is compared with the values of each case,
  - If there is a match, the associated block of code is executed,
  - The break keyword is used to skip subsequent cases.
  - The default keyword is used to jump to the default case

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

`switch-case` statement

- Flowchart for the switch-case statement shown on the right.
- The expression must evaluate to integer or constant values.
- Floating point or double precision values are not allowed.
- enumerated values (**`enum`**) may also be used for the expression.





switch-case Examples

- Design and implement a program in C++ that uses a switch statement to state whether an input integer is odd, even, prime or composite. Try using if-else for this program.
- The input must be between 2 and 9 (inclusive), else a default statement “out of range” is printed. Try using switch also.

```cpp
int main() {
    int num;
    cout << "Enter an integer between 2 and 9: ";
    cin >> num;

    switch (num) {
        case 2:
            cout << num << " is even and prime." << endl;
            break;
        case 3:
        case 5:
        case 7:
            cout << num << " is odd and prime." << endl;
            break;
        case 4:
        case 6:
        case 8:
            cout << num << " is even and composite." << endl;
            break;
        case 9:
            cout << num << " is odd and composite." << endl;
            break;
        default:
            cout << "Out of range." << endl;
    }
    return 0;
}
```

Switch-case Examples using enum

```cpp
enum Months {
    January = 1, February, March, April, May, June,
    July, August, September, October, November, December
};

int main() {
    int month;
    cout << "Enter the month (1-12): ";
    cin >> month;

    switch (month) {
        case January: case March: case May: case July:
        case August: case October: case December:
            cout << "31 days." << endl;
            break;
        case April: case June: case September: case November:
            cout << "30 days." << endl;
            break;
        case February:
            cout << "28 or 29 days (in a leap year)." << endl;
            break;
        default:
            cout << "Invalid month. Please enter a number between 1 and 12." << endl;
    }
    return 0;
}
```

Arrays

- An array is a data type containing elements of the same type (homogeneous) data, stored in contiguous memory locations.
- We declare an array by specifying the data type, followed by the name of the array, followed by its size in (square brackets).
- `int list[4]` declares an array of 4 integers called `list`.

list[0]	list[1]	list[2]	list[3]

- Each element of the array can be accessed by its subscripts.
- Subscripts start from 0 to size – 1 (in this case from 0 to 3).

```
//Some array
//initialization options

int a[4];
//declares an
//array of size 4

int a[] = {20, 40};
//declares an array
//with values
//size is implicit

int a[];
//illegal; size must
//be specified
```

Arrays in C++

- Arrays in C++ can be one-dimensional or multi-dimensional.
- A declaration of a two dimensional array is given as follows:
- `double m[3][4];` *//this declares an array of size 12 i.e. 3 rows and 4 columns.*
- The array elements are indexed as follows:

m[0][0]	m[0][1]	m[0][2]	m[0][3]
m[1][0]	m[1][1]	m[1][2]	m[1][3]
m[2][0]	m[2][1]	m[2][2]	m[2][3]

```
//LEGAL WAYS TO
//DECLARE A 2-D ARRAY
int m[][4] = {{1, 2, 3, 4},
              {5, 6, 7, 8},
              {9, 10, 11, 12}};

int m[3][4];
```

Arrays in C++

- Some completely legal but weird ways of initializing arrays

```
// 1. weird array initializations that work
int arr0[2][3] = { {1, 2}, {3} }; // {{1,2,0},{3,0,0}}
int arr1[2][3] = {1, 2, 3, 4}; // {{1,2,3},{4,0,0}}
int arr2[]{1, 2, 3};   // Size deduced: 3
int arr3[5]{};         // All zeros: {0, 0, 0, 0, 0} also int arr3[5] = {}
```

Processing arrays in C++

- We use loops (e.g. for-loops) to process arrays in C++. To print a 1-D array:
- ```
  int a[] = {1, 2, 3, 4};
  for(int i = 0; i < 4; i++)
      cout << a[i] << " ";
  ```
- To process a 2-D array, we use nested for-loops. To print a 2-D array:
- ```
  int a[][2] = {{1, 2}, {3, 4}};
      for(int i = 0; i < 2; i++) {
          for(int j = 0; j < 2; j++)
              cout << a[i][j] << " ";
          cout << "\n";
      }
  ```

Arrays and Functions (Without pointers)

- Arrays can be passed as parameters to a function.
- However, the function has to be declared first.
- This is necessary before the first use of the function in the main function.
- A function declaration can be done by declaring the function by stating its return type, followed by name, followed by the signatures (parameters and their types).
- `int power(int base, into exponent)` is a function declaration.
- In C++ a function definition (the header and entire body of the function) can also serve as a declaration, if it is stated before the main function.
- Examples of function declaration with arrays:

Arrays and Functions

```
#include <iostream>
#include <iomanip>              //for setprecision, etc.
using namespace std;
const int DISTRICTS = 4;    //array dimensions
const int MONTHS = 3;
void display( double[DISTRICTS][MONTHS] );  //declaration
//-----------------------------------------------------------
int main()
   {                            //initialize two-dimensional array
   double sales[DISTRICTS][MONTHS]
```

- Observe the use of `const` to declare constants `DISTRICTS` and `MONTHS`

Arrays and Functions

Function Declaration with Array Arguments

In a function declaration, array arguments are represented by the data type and size of the array. Here's the declaration of the `display()` function:

```
void display( float[DISTRICTS][MONTHS] );  // declaration
```

Actually, there is one unnecessary piece of information here. The following statement works just as well:

```
void display( float[][MONTHS] );  // declaration
```

It follows that if we were declaring a function that used a one-dimensional array as an argument, we would not need to use the array size:

```
void somefunc( int elem[] );   // declaration
```

Arrays and Functions – function calls

Function Call with Array Arguments

When the function is called, only the name of the array is used as an argument.

```
display(sales);   // function call
```

Example: 1-D Array Addition and Multiplication. Write a program in C++ that reads in as inputs two arrays of one dimension using a function read. It then computes the element wise sum and product of these two arrays using the functions sum and product. Make sure to use suitable parameter lists for the functions.

Arrays and Functions

- In an a function declaration, all (but one) of the subscripts of the array size must be stated.
- So for example,
- `void f(int [][] a)` is not legal, but `void f(int a[][COLS])` is legal.
- Likewise for a 3 dimensional array, `void f(int a[][WIDTH][DEPTH])` is legal.
- It is advisable to pass the length of the array as a parameter, or declare it as a global constant using `#define` primitive.

1-D Arrays as Strings

- Two kinds of strings used in C++:
  - Objects of the `string` class.
  - One dimensional arrays of type `char`, also known as C-strings.
- A C-string can be defined as an array of characters.
- The length of the string has to be declared.
- Usually it is the maximum possible size the string is supposed to have.
- Here's one way to declare a C-string:

```
const int MAX = 80;
char str[MAX];
```

Summary

- In this slideset we studied switch statements with options for default, break.
- We also studied arrays and parameter passing using arrays and functions.
- Finally we studied the use of 1-D arrays as strings.
- It needs to be mentioned here that C++ does not have methods for checking array bounds, i.e. accessing an element beyond the array boundaries is not checked.
- For this reason, due care should be exercised while accessing array elements.